Autenticación con tokens en Café Nube

Erian Badilla Fallas · 5 de noviembre de 2026

Café Nube incorporará registro, inicio de sesión y acceso autenticado a sus seis módulos. Se conservarán MySQL, Sequelize y los datos existentes. Esta investigación precede a la implementación de autenticación y fundamenta las decisiones del Día 2.

JWT y verificación

Un JWT representa afirmaciones sobre una identidad. En el formato firmado JWS compacto tiene tres segmentos separados por puntos: encabezado, contenido y firma. El encabezado identifica el algoritmo; el contenido contiene las afirmaciones; la firma permite comprobar integridad y autenticidad. La codificación del contenido no lo vuelve secreto. Nunca incluiremos contraseñas ni hashes en él [1].

El servidor emitirá el token con jsonwebtoken después de comprobar las credenciales. Usará HS256, una clave aleatoria privada y una duración de una hora. El contenido incluirá el identificador en sub, el emisor, la audiencia, la fecha de emisión y la de expiración. La verificación fijará el algoritmo permitido y comprobará firma, emisor, audiencia y vencimiento. Decodificar sin verificar no autentica al usuario [2, 3].

Vencimiento y renovación

Al vencer, el servidor rechazará el token con 401. React limpiará la sesión y mostrará el login. No se extenderá el tiempo modificando el contenido en el navegador.

Un refresh token permite solicitar nuevos tokens de acceso sin volver a introducir la contraseña. Necesita almacenamiento y reglas propias, como rotación o vinculación al cliente, detección de reutilización y revocación. El enunciado exige investigar este mecanismo, pero no implementarlo. Para el alcance académico elegimos volver a iniciar sesión al cumplirse la hora [4].

Dónde guardar el token

Alternativa	Ventaja	Riesgo o condición
Memoria de React	No persiste como dato del navegador	Se pierde al recargar; XSS activo puede actuar como el usuario
localStorage	Persiste entre visitas	JavaScript puede leerlo; un XSS puede robarlo
sessionStorage	Persiste al recargar dentro de la sesión de la pestaña	También es accesible a JavaScript y vulnerable a XSS
Cookie HttpOnly	JavaScript no puede leer directamente el token	El navegador la envía automáticamente; requiere estudiar CSRF, SameSite, Secure y la topología del despliegue

La decisión final del proyecto es utilizar una cookie HttpOnly, host-only y con Path=/api para guardar el JWT. El navegador la envía mediante fetch con credentials: include; React no recibe el JWT en el JSON ni lo guarda en Web Storage. Se conserva el perfil en AuthContext. HttpOnly impide leer la cookie desde JavaScript, pero no impide que un XSS activo actúe con la sesión [7].

La cookie usa SameSite=Lax por defecto y vence en una hora. En producción lleva Secure y requiere HTTPS. Para el desarrollo local con HTTP, Secure permanece desactivado. Frontend y API deben usar el mismo nombre de host, por ejemplo localhost en ambos; cambiar de puerto no cambia el sitio. Si se despliegan en sitios distintos, se deben revisar SameSite=None, Secure y las restricciones de cookies de terceros [7].

Flujo en Express y React

El registro valida nombre, correo único y contraseña; guarda únicamente su hash. El login compara ese hash, coloca el JWT en una cookie HttpOnly y devuelve el usuario sin password. Un middleware verifica el JWT recibido en la cookie HttpOnly y consulta el usuario identificado antes de permitir la operación. Registro y login son públicos; el perfil, el resumen y los CRUD quedan protegidos.

AuthContext conserva el estado de sesión. Una función común de fetch incorpora credentials: include. RutaProtegida espera la comprobación del perfil antes de mostrar contenido. Un 401 limpia el estado de React y redirige al login. Cerrar sesión llama a POST /api/auth/logout y el servidor elimina las cookies con su mismo nombre y alcance. Una copia externa del JWT seguirá siendo válida hasta vencer; no se promete revocación remota inmediata.

Protección CSRF

Como las cookies se envían automáticamente, se exige X-CSRF-Token en todas las escrituras, incluidos registro, login y logout. GET /api/auth/csrf entrega un valor HMAC calculado con el secreto del servidor y ligado al JWT actual. Antes del login utiliza un nonce aleatorio en otra cookie HttpOnly. El servidor compara el valor en tiempo constante y rechaza los orígenes ajenos. Un valor de otra sesión no permite modificar datos; SameSite es una defensa adicional, no el único control [10].

Contraseñas y configuración

Se usará bcrypt con coste 12 y sal generada por la biblioteca. El registro exigirá al menos ocho caracteres y rechazará entradas que superen 72 bytes UTF-8 para evitar el truncamiento del algoritmo. El hash no se devuelve en respuestas ni se registra en logs [8, 9].

JWT_SECRET quedará solo en backend/.env y se generará de forma aleatoria durante la configuración. El ejemplo permanecerá vacío y el servidor rechazará un secreto ausente o demasiado corto. CORS permitirá credenciales, Content-Type y X-CSRF-Token desde orígenes exactos. Las pruebas locales usarán HTTP en loopback; un despliegue debe emplear HTTPS. Se limitarán los intentos en registro y login. Todos los usuarios autenticados tendrán el mismo acceso, como pide el enunciado.

Fuentes consultadas

Consulta realizada el 5 de noviembre de 2026.

1. IETF. RFC 7519 JSON Web Token. https://www.rfc-editor.org/rfc/rfc7519

2. IETF. RFC 8725 JSON Web Token Best Current Practices. https://www.rfc-editor.org/rfc/rfc8725.html

3. Auth0. node-jsonwebtoken README. https://github.com/auth0/node-jsonwebtoken

4. IETF. RFC 9700 Best Current Practice for OAuth 2.0 Security, sección 4.14. https://www.rfc-editor.org/rfc/rfc9700.html

5. OWASP. HTML5 Security Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

6. MDN. Window sessionStorage. https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage

7. MDN. Set-Cookie. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie

8. Kelektiv. node.bcrypt.js. https://github.com/kelektiv/node.bcrypt.js

9. OWASP. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

10. OWASP. Cross-Site Request Forgery Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html